

Solver Specification:

highOrderElectroActivationFoamImplicitJfNK

A high-order, implicit, Jacobian-free Newton–Krylov monodomain solver in OpenFOAM with an Eigen linear-algebra back-end

Pablo

July 5, 2026

Abstract

This report specifies the algorithms implemented in the OpenFOAM solver `highOrderElectroActivationFoamImplicitJfNK`. The solver advances the monodomain equations of cardiac electrophysiology coupled to the ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004) ionic cell model. The diffusion operator is discretised with the cell-centred finite-volume framework of OpenFOAM, optionally upgraded to a Local Radial-Extrapolation (LRE) high-order reconstruction; the ionic source is evaluated either at cell centres or at LRE cell quadrature points, with the reaction ODE integrated at the quadrature points, once per cell and reconstructed, or by a front-aware hybrid that does the former only on the propagating front and the latter elsewhere (Section 5); time integration follows a θ -scheme (backward Euler or Crank–Nicolson); the resulting nonlinear coupled system can be solved by Picard fixed-point iteration, a diagonal I_{ion} linearisation, or a Jacobian-Free Newton–Krylov (JFNK) method with Armijo backtracking line search, linear extrapolation initial guess and a physical-bracket clamp on the input to the ionic ODE; the JFNK inner linear system is solved by an in-house restarted GMRES with modified Gram–Schmidt orthogonalisation built on top of the Eigen sparse linear algebra library; the per-cell ionic ODE uses OpenFOAM’s adaptive RKF45 integrator with configurable tolerances; activation times are tracked by linear interpolation across the first threshold crossing and benchmark points are sampled by inverse-distance weighting. All algorithmic parameters are exposed through the `constant/spatialIntegrationProperties` and `constant/timeIntegrationProperties` dictionaries, and the companion Python driver `run_convergence.py` generates these dictionaries from a single configuration file. This solver is algorithmically identical to its PETSc-based sibling `highOrderElectroActivationFoamImplicitJfNK_PETSc`; the only difference is the linear-algebra library (Eigen vs. PETSc).

Contents

1	Notation	2
2	Governing equations	2
2.1	Monodomain PDE	2
2.2	Ionic ODE	2
2.3	Manufactured MMS equation used for convergence transfer	3
2.4	External stimulus	3
3	Time discretisation: the θ-scheme	3
4	Spatial discretisation	4
4.1	Standard orthogonal FV Laplacian	4
4.2	High-order LRE diffusion	4
4.3	Mass matrices	4
4.4	High-order reconstruction at ionic quadrature points	4
4.5	High-order cell averaging of I_{ion}	5

5	State integration at the ionic quadrature points	5
5.1	Two modes	5
5.2	Why the reaction ODE must be solved at the Gauss points on the front	5
5.3	Front-aware hybrid	6
5.4	State reconstruction limiter	7
5.5	Dictionary keys	7
6	Nonlinear solver	7
6.1	Shared residuals and stopping tests	7
6.2	Picard fixed-point method	8
6.3	Diagonal I_{ion} linearisation	8
6.4	Jacobian-Free Newton–Krylov	9
6.5	In-house restarted GMRES	9
6.6	Armijo backtracking line search	9
6.7	Initial guess, ODE clamp and rollback	9
7	Linear solver: Eigen back-end	10
8	Ionic ODE integration	10
9	TNNP numerical protection	10
10	Activation tracking and benchmarks	11
11	Solver flow chart	11
12	Manufactured driver: run_convergence.py	13
12.1	Geometry and parameter sweep	13
12.2	Nonlinear method and convergence controls	14
12.3	JFNK and diagonal-Ion controls	14
12.4	Linear PDE solver	15
12.5	State ODE integrator	15
12.6	LRE high-order parameters per p-level	16
12.7	Diagnostics and run-control	16
13	Results	16
13.1	2D Niederer-style benchmark	16
13.2	3D Niederer-style benchmark	18
A	Dimension table	19
B	File map	20
C	Differences vs. the PETSc sibling	20

1 Notation

Symbol	Meaning	Units
$\Omega \subset \mathbb{R}^d$	Spatial domain ($d \in \{2, 3\}$)	m
$\partial\Omega$	Insulated boundary (zero-flux Neumann)	—
$t \in (0, T]$	Time	s
$V_m(\mathbf{x}, t)$	Transmembrane voltage	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	TNNP ionic states	mixed
$I_{\text{ion}}(V_m, \mathbf{s})$	Total ionic current density	V s^{-1}
$I_{\text{ext}}(\mathbf{x}, t)$	External stimulus current density	A m^{-3}
χ	Membrane surface-to-volume ratio	m^{-1}
C_m	Membrane capacitance per unit area	F m^{-2}
$\mathbf{D}(\mathbf{x})$	Bulk conductivity tensor	S m^{-1}
$\theta \in \{0.5, 1\}$	Time-stepping parameter (CN or BE)	—
Δt	Outer time-step length	s
N_c	Number of mesh cells	—
$\mathbf{V}^n \in \mathbb{R}^{N_c}$	Vector of cell-averaged V_m at time t^n	V
$\mathbf{M}, \mathbf{K}, \mathbf{L}$	Mass, stiffness, scaled diffusion matrix	—
$\mathbf{A}, \mathbf{B}$	Implicit LHS / RHS operators of the θ -scheme	—
$\mathbf{F}(\mathbf{x})$	Discrete nonlinear residual to be zeroed	—

2 Governing equations

2.1 Monodomain PDE

The bulk voltage is propagated by the standard monodomain reaction–diffusion equation [1, 2]. In physical current-density notation one may write

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi C_m I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}} \quad \text{in } \Omega \times (0, T], \quad (1)$$

subject to insulated boundaries $\mathbf{n} \cdot (\mathbf{D} \nabla V_m) = 0$ on $\partial\Omega \times (0, T]$, and initial condition $V_m(\mathbf{x}, 0) = -84 \text{ mV}$. The conductivity tensor $\mathbf{D}$ is symmetric positive-definite and either uniform or read as a `volTensorField` from disk. The factor χC_m multiplying I_{ion} in (1) reflects the convention used by the ionic-model interface: the solver stores `Iion` as a voltage rate with dimensions V s^{-1} , not as an A m^{-3} volumetric current.

Dividing (1) by χC_m gives the form actually used inside the solver,

$$\frac{\partial V_m}{\partial t} = \mathcal{L} V_m + s(V_m, \mathbf{s}), \quad \mathcal{L} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla \cdot), \quad s = -I_{\text{ion}} + \frac{I_{\text{ext}}}{\chi C_m}. \quad (2)$$

This is exactly what `rhsFromIion` assembles in the solver.

2.2 Ionic ODE

At every spatial sampling point the TNNP state vector $\mathbf{s} \in \mathbb{R}^{17}$ evolves according to

$$\frac{d\mathbf{s}}{dt} = \mathbf{f}_{\text{TNNP}}(V_m, \mathbf{s}; \text{tissue}), \quad \mathbf{s}(0) = \mathbf{s}_0, \quad (3)$$

where `tissue` selects `epicardialCells`, `mCells` or `endocardialCells`. The 17 tracked states are

$$\mathbf{s} = (V, K_i, Na_i, Ca_i, Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g, Ca_{SR}).$$

The total ionic current returned to the PDE is the algebraic `Iion.cm` of TNNP [1].

2.3 Manufactured MMS equation used for convergence transfer

The convergence driver in `tutorials_electro/highOrderManufacturedFDAImplicitJfNK` solves a manufactured problem with the same numerical structure as the electro-activation solver: implicit monodomain-like PDE, local state ODEs, nonlinear ionic source, optional LRE reconstruction for V_m and I_{ion} , and selectable `Picard/diagonalIon/JfNK` nonlinear coupling. The analytical fields are

$$V_m^*(\mathbf{x}, t) = \sqrt{1+t} F(\mathbf{x}), \quad (4)$$

$$u_1^*(\mathbf{x}, t) = (1+t) G(\mathbf{x}) + V_m^*(\mathbf{x}, t), \quad (5)$$

$$u_2^*(\mathbf{x}, t) = \frac{1}{(1+t)\sqrt{G(\mathbf{x})}}, \quad (6)$$

$$u_3^*(\mathbf{x}, t) = 0, \quad (7)$$

with, for example, $F = \cos(\pi x) \cos(2\pi y)$ and $G = 1 + x y^2$ in 2D. In 3D, $F = \cos(\pi x) \cos(2\pi y) \cos(3\pi z)$ and $G = 1 + x y^2 z^3$. The state equations are

$$\dot{u}_1 = (u_1 + u_3 - V_m)^2 u_2^2 + \frac{1}{2}(u_1 + u_3 - V_m) u_2^2 (V_m - u_3), \quad (8)$$

$$\dot{u}_2 = -(u_1 + u_3 - V_m) u_2^3, \quad (9)$$

$$\dot{u}_3 = 0. \quad (10)$$

The manufactured ionic source is

$$I_{\text{ion}}^{\text{MMS}}(V_m, \mathbf{u}) = -\frac{1}{2}(u_1 + u_3 - V_m) u_2^2 (V_m - u_3) + \frac{\beta}{\chi C_m} (V_m - u_3), \quad (11)$$

and the PDE source used by the manufactured solver is $-I_{\text{ion}}^{\text{MMS}}$. The coefficient β is the spatial eigenvalue of the trigonometric manufactured field under the constant conductivity tensor:

$$\beta = \begin{cases} -\pi^2 D_{xx}, & d = 1, \\ -\pi^2 (D_{xx} + 4D_{yy}), & d = 2, \\ -\pi^2 (D_{xx} + 4D_{yy} + 9D_{zz}), & d = 3. \end{cases}$$

Thus the MMS case is not intended to reproduce TNNP physiology; it is a controlled problem with a known exact solution that exercises the same discrete operators and nonlinear solution paths used by this solver.

2.4 External stimulus

The external current is a time- and region-windowed step:

$$I_{\text{ext}}(\mathbf{x}, t) = \begin{cases} I_0 & \mathbf{x} \in \Omega_{\text{stim}}^{(b)}, \quad t \in [t_0^{(b)}, t_0^{(b)} + \tau], \quad b = 1, \dots, B, \\ 0 & \text{otherwise,} \end{cases} \quad (12)$$

with $I_0 = 50\,000 \text{ A m}^{-3}$ and $\tau = 2 \text{ ms}$ as Niederer-benchmark defaults.

3 Time discretisation: the θ -scheme

Time is advanced from t^n to $t^{n+1} = t^n + \Delta t$ by

$$\frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{\Delta t} = \theta \mathbf{L} \mathbf{V}^{n+1} + (1 - \theta) \mathbf{L} \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n. \quad (13)$$

implicitScheme	θ	Scheme
backwardEuler	1	First-order, L-stable
crankNicolson	1/2	Second-order, A-stable

After multiplication by Δt^{-1} the implementation forms

$$[\mathbf{M}/\Delta t - \theta \mathbf{L}] \mathbf{V}^{n+1} = [\mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}] \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n, \quad (14)$$

with the two Eigen `SparseMatrix<scalar, RowMajor>` operators

$$\mathbf{A} = \mathbf{M}/\Delta t - \theta \mathbf{L}, \quad \mathbf{B} = \mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}. \quad (15)$$

$\mathbf{A}$ and $\mathbf{B}$ are rebuilt at every time step by copying $\mathbf{M}$, scaling by Δt^{-1} and adding $\pm \theta \mathbf{L}$. When $\theta = 1$ (BE) the explicit half of the diffusion vanishes and $\mathbf{B} = \mathbf{M}/\Delta t$. The scaled diffusion is $\mathbf{L} = \mathbf{K}/(\chi C_m)$.

4 Spatial discretisation

4.1 Standard orthogonal FV Laplacian

`assembleStandardOrthogonalStiffnessMatrix` computes the classical 5/7-point Laplacian. Per internal face f with face area $\mathbf{S}_f$,

$$a_f = \frac{|\mathbf{S}_f| (\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \frac{1}{2} (\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}), \quad (16)$$

contributing $-a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own}, \text{own}]$, $+a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own}, \text{nei}]$, and the transposed pair to the neighbour row. `zeroGradient/empty` patches contribute zero flux. `fixedValue` patches contribute to the owner diagonal only.

4.2 High-order LRE diffusion

When `useHighOrder_Vm = true` and $d > 1$, `assembleHighOrderStiffnessMatrix` integrates the flux on a face quadrature with reconstructed gradients:

$$\Phi_f = \sum_{q \in \mathcal{Q}_f} w_q |\mathbf{S}_f| \mathbf{n}_f \cdot \mathbf{D}_f \cdot \left(\sum_{j \in \mathcal{S}_f} \mathbf{g}_{f,q,j} \mathbf{V}_j \right), \quad (17)$$

where $\mathcal{S}_f$ is the LRE face stencil (~ 15 – 50 cells) and $\mathbf{g}_{f,q,j} = \mathbf{QRGradFaceGPCoeffs}$. The contribution of each $\mathbf{V}_j$ is added to row `own` with weight $+\Phi_{f,j}/V_{\text{own}}$ and to row `nei` with weight $-\Phi_{f,j}/V_{\text{nei}}$, yielding a negative semi-definite Laplacian.

4.3 Mass matrices

Two options:

Lumped (`massMatrix lumped`). `assembleDiagonalMassMatrix` writes $\mathbf{M} = \mathbf{I}$.

Consistent HO (`massMatrix consistent`). `assembleConsistentMassMatrixHO` fills

$$\mathbf{M}[i, j] = \sum_{q \in \mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}} \phi_j(\mathbf{x}_q), \quad \phi_j(\mathbf{x}_q) = \delta_{ij} + \mathbf{g}_{i,j} \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_{i,j} : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_{i,j}^{(3)} : (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (18)$$

with $\mathbf{d}_q = \mathbf{x}_q - \mathbf{C}_i$. A safety guard silently downgrades to the lumped form when `useHighOrder_Vm = false` or $d \leq 1$.

4.4 High-order reconstruction at ionic quadrature points

If `useHighOrder_Iion = true`, the ionic ODE is integrated at cell quadrature points. $\mathbf{V}_m$ is reconstructed there by the same Taylor expansion as in (18):

$$V_m(\mathbf{x}_q) \approx V_{m,c} + (\nabla V_m)_c \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_c : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_c^{(3)} : (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (19)$$

implemented in `reconstructVmAtIionIntegrationPoints`.

4.5 High-order cell averaging of I_{ion}

`averageIionIntegrationPointsToCells` contracts the per-QP I_{ion} back to cells:

$$I_{\text{ion},i} = \frac{\sum_{q \in \mathcal{Q}_i} w_q I_{\text{ion},q}}{\sum_{q \in \mathcal{Q}_i} w_q}. \quad (20)$$

5 State integration at the ionic quadrature points

The high-order quadrature (19) evaluates $I_{\text{ion}}(V_m, \mathbf{s})$ at the cell quadrature points $\mathbf{x}_q$, so it needs the ionic state vector $\mathbf{s}$ there as well. *How* the states reach the quadrature points is governed by `stateIntegrationMode` and, orthogonally, by `frontAwareHybrid`. This choice turns out to be decisive for a propagating cardiac front, and this section documents both the algorithm and the reason it matters.

5.1 Two modes

Let $\mathcal{Q}_i = \{\mathbf{x}_q\}$ be the quadrature set of cell Ω_i with weights w_q , and let $\mathbf{s}_c$ denote a cell-centred state.

gaussPointODE (legacy). The stiff ODE (3) is integrated *independently at every quadrature point*, driven by the locally reconstructed potential:

$$\mathbf{s}_q^{n+1} = \text{ODEsolve}(\mathbf{s}_q^n, V_m(\mathbf{x}_q); \Delta t), \quad I_{\text{ion},q} = \phi(V_m(\mathbf{x}_q), \mathbf{s}_q^{n+1}). \quad (21)$$

Cost: N_q stiff integrations per cell.

cellCentredReconstruct. The ODE is integrated *once per cell* at the cell centre, and the resulting cell-centred states are reconstructed to the quadrature points by a dedicated LRE (`LREInterp_states`, order set by `highOrderCoeffs/LRECoeffs_states`):

$$\mathbf{s}_c^{n+1} = \text{ODEsolve}(\mathbf{s}_c^n, \bar{V}_{m,c}; \Delta t), \quad \hat{\mathbf{s}}(\mathbf{x}_q) = \mathcal{R}^{(p)}[\{\mathbf{s}_c^{n+1}\}](\mathbf{x}_q), \quad I_{\text{ion},q} = \phi(V_m(\mathbf{x}_q), \hat{\mathbf{s}}(\mathbf{x}_q)). \quad (22)$$

Cost: one stiff integration per cell – a factor N_q cheaper, which for the expensive TNNP integrations is the motivation. The two modes differ only in the argument of the non-linear map $V_m \mapsto \mathbf{s}$: pointwise $V_m(\mathbf{x}_q)$ in (21), versus the cell average $\bar{V}_{m,c}$ in (22).

5.2 Why the reaction ODE must be solved at the Gauss points on the front

For a smooth reaction field the two modes agree to the reconstruction order, and `cellCentredReconstruct` is preferable for its speed. On a *propagating cardiac front* they do not agree, and only the pointwise evaluation (21) is admissible. The reason is the strong non-linearity of the sodium activation.

The upstroke is carried by $I_{\text{Na}} = g_{\text{Na}} m^3 h j (V_m - E_{\text{Na}})$, whose activation gate relaxes towards the steep sigmoid

$$m_{\infty}(V_m) = \left[1 + \exp((V_{1/2} - V_m)/k) \right]^{-1}, \quad V_{1/2} \approx -40 \text{ mV}, \quad k \approx 6 \text{ mV}. \quad (23)$$

Below its inflection ($V_m < V_{1/2}$) – exactly the sub-threshold foot where ignition and conduction are decided – m_{∞} is *convex*. Writing $\langle \cdot \rangle_i = (\sum_q w_q \cdot) / \sum_q w_q$ for the cell-quadrature average, so that $\bar{V}_{m,i} = \langle V_m \rangle_i$, the two modes evaluate the gate as

$$\underbrace{\langle m_{\infty}(V_m) \rangle_i}_{\text{gaussPointODE}} \geq \underbrace{m_{\infty}(\langle V_m \rangle_i)}_{\text{cellCentredReconstruct}}, \quad (24)$$

by Jensen’s inequality for the convex m_{∞} . Since $I_{\text{Na}} \propto m^3$, the gap is amplified cubically: the cell-average path *systematically under-estimates* the regenerative inward current precisely where V_m varies within a cell, i.e. at the wavefront and the stimulus edge. Reconstructing the states after the ODE (as in (22)) cannot repair this – the bias is committed *inside* the ODE, whose input was already the averaged potential.

A quantitative estimate. Let $\Delta V = \max_q V_m(\mathbf{x}_q) - \min_q V_m(\mathbf{x}_q)$ be the sub-cell range of V_m across a cell straddling the foot of the upstroke, and treat V_m as roughly uniform over the cell (variance $\sigma^2 \simeq \Delta V^2/12$). A midpoint expansion of the average gives

$$\langle m_\infty \rangle - m_\infty(\bar{V}) \simeq \frac{1}{2} m_\infty''(\bar{V}) \sigma^2 = m_\infty''(\bar{V}) \frac{\Delta V^2}{24}. \quad (25)$$

On the exponential foot $m_\infty \approx e^{(V_m - V_{1/2})/k}$, so $m_\infty''/m_\infty = 1/k^2$ and, using $I_{\text{Na}} \propto m^3$,

$$\boxed{\frac{\delta I_{\text{Na}}}{I_{\text{Na}}} \approx 3 \cdot \frac{\Delta V^2}{24 k^2} = \frac{\Delta V^2}{8 k^2}}. \quad (26)$$

With $k = 6 \text{ mV}$ and a sub-cell range $\Delta V \sim 30 \text{ mV}$ across an under-resolved front, (26) gives $\delta I_{\text{Na}}/I_{\text{Na}} \approx 900/288 \approx 3$ – the linearised estimate is already $O(1)$, i.e. the cell-average evaluation under-shoots the sodium current by a factor of order unity or more. Two failure modes follow, both observed and documented in the companion note `stateIntegrationMode.conductionDiagnosis.tex`:

1. **Ignition failure:** a nominal stimulus is rendered effectively sub-threshold; V_m rises but the regenerative upstroke never fires.
2. **Conduction block:** once ignited, each front cell supplies too little inward charge to bring its downstream neighbours to threshold – a numerically induced source–sink mismatch that stalls the wave.

Refining Δt makes the front sharper (larger ΔV) and thus *aggravates* the deficit, which is why the pure `cellCentredReconstruct` path fails at small Δt while a coarse- Δt over-shoot can mask it. The conclusion is unavoidable: at high order on a sharp front the reaction ODE *must* be solved at the Gauss points of the front cells; only the remaining (smooth) volumes may be reconstructed cheaply.

5.3 Front-aware hybrid

`frontAwareHybrid` (default `false`, orthogonal to `stateIntegrationMode`) realises exactly that conclusion: it solves the ODE at the Gauss points in the thin band of *front* cells and at the cell centre (with reconstruction) everywhere else. A cell i is a *front* cell when its sub-cell V_m range exceeds `hybridFrontVmRange` or it is stimulated,

$$\text{front}_i \iff \left(\max_q V_m(\mathbf{x}_q) - \min_q V_m(\mathbf{x}_q) \right) > \text{hybridFrontVmRange} \vee (I_{\text{ext}})_i \neq 0. \quad (27)$$

The persistent states are carried at the Gauss points (as in `gaussPointODE`); the smooth-cell reconstruction derives its cell values by quadrature-averaging that carrier. Per nonlinear evaluation the branch is:

- (a) front cells: one ODE per Gauss point, local V_m (as (21));
- (b) smooth cells: one ODE at the cell centre from the cell-average state;
- (c) reconstruct the smooth-cell states to their Gauss points, front Gauss states left untouched;
- (d) evaluate I_{ion} at all Gauss points and average to the cell.

Freezing per time step (avoiding a limit cycle). The front classification (27) is a *solution-dependent discrete switch*. If recomputed inside the nonlinear loop, cells near the threshold flip $\text{front} \leftrightarrow \text{smooth}$ between iterations; because the two branches compute I_{ion} by different paths, the source term jumps discretely and the Picard fixed-point map becomes discontinuous, producing a non-convergent *period-2 limit cycle* (observed as $\text{iter}_{k+2} \equiv \text{iter}_k$ in `nonlinearResiduals.dat`). The classification is therefore computed *once per time step* from the committed V_m^n and *frozen* for the whole nonlinear solve, and the band is dilated by `hybridFrontDilation` cell layers (via `mesh.cellCells()`) so the front stays inside the Gauss-resolved zone as it advances. Freezing is safe because the front moves far less than one cell per step (for the 2D benchmark, $\sim 0.09 \text{ mm/ms}$, i.e. ~ 0.009 cell per step at $\Delta t = 10^{-4} \text{ s}$, $\Delta x = 0.5 \text{ mm}$).

5.4 State reconstruction limiter

High-order ($p3$) reconstruction of the stiff TNNP states across an under-resolved front overshoots (Gibbs), and a state driven far outside its physical range can overflow the ionic algebra (observed as a PETSc SIGFPE). `stateReconstructLimiter` (default `true`, active for `cellCentredReconstruct` and the hybrid smooth cells, independent of `frontAwareHybrid`) applies a Barth–Jespersen-style bound: each reconstructed value is clipped to the range of the cell and its face neighbours,

$$\widehat{s}(\mathbf{x}_q) \leftarrow \text{clip}\left(\widehat{s}(\mathbf{x}_q), \min_{c' \in \{i\} \cup \mathcal{N}(i)} s_{c'}, \max_{c' \in \{i\} \cup \mathcal{N}(i)} s_{c'}\right). \quad (28)$$

No new extremum is created, so the reconstructed states stay within the physically realised (ODE-evolved) range and the ionic algebra never sees a pathological input; in smooth regions the reconstruction already lies within the range and the limiter is inactive, preserving the order there.

5.5 Dictionary keys

Key (in <code>implicitHOCoeffs</code>)	Meaning	Default
<code>stateIntegrationMode</code>	<code>gaussPointODE</code> / <code>cellCentredReconstruct</code>	<code>cellCentredReconstruct</code>
<code>frontAwareHybrid</code>	Gauss ODEs on the front, reconstruct elsewhere	<code>false</code>
<code>hybridFrontVmRange</code>	sub-cell V_m range flagging a front cell [V]	5×10^{-3}
<code>hybridFrontDilation</code>	cell layers added around the front band	2
<code>stateReconstructLimiter</code>	Barth–Jespersen limiter on the reconstruction	<code>true</code>

`LRECoeffs_states` (in `highOrderCoeffs`) sets the reconstruction order; it is required by `cellCentredReconstruct` and by the hybrid.

6 Nonlinear solver

At each outer time step the unknown is the new voltage vector $\mathbf{x} = \mathbf{V}^{n+1}$. For any candidate $\mathbf{x}$ the solver first reconstructs the corresponding V_m field, then calls `evaluateNonlinearFields`. That routine resets the ionic states to their time-step-start values $\mathbf{s}^n$, integrates the local ODEs from t^n to t^{n+1} using the candidate voltage, obtains $I_{\text{ion}}(\mathbf{x})$, and optionally averages quadrature-point values back to cells. This gives the nonlinear residual

$$\mathbf{F}(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{r}(I_{\text{ion}}(\mathbf{x})), \quad \mathbf{x} = \mathbf{V}^{n+1}, \quad (29)$$

with

$$\mathbf{r}(I_{\text{ion}}) = \mathbf{B}\mathbf{V}^n + \theta \left[-I_{\text{ion}}(\mathbf{x}) + I_{\text{ext}}^{n+1}/(\chi C_m) \right] + (1 - \theta) \left[-I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m) \right]. \quad (30)$$

The three nonlinear methods differ only in how they drive $\mathbf{F}(\mathbf{x})$ toward zero.

6.1 Shared residuals and stopping tests

Every method records the same diagnostics:

$$R_{\text{cpl}} = \frac{\|\mathbf{F}(\mathbf{x}_k)\|_2}{\|\mathbf{x}_k\|_2 + \epsilon}, \quad (31)$$

$$R_{V_m} = \frac{\|\mathbf{V}_k - \mathbf{V}_{k-1}\|_2}{\|\mathbf{V}_{k-1}\|_2 + \epsilon}, \quad (32)$$

$$R_{I_{\text{ion}}} = \frac{\|I_{\text{ion},k} - I_{\text{ion},k-1}\|_2}{\|I_{\text{ion},k-1}\|_2 + \epsilon}, \quad (33)$$

$$R_s = \max_j \frac{\|\mathbf{s}_{j,k} - \mathbf{s}_{j,k-1}\|_2}{\|\mathbf{s}_{j,k-1}\|_2 + \epsilon}. \quad (34)$$

The actual implementation uses `relativeL2Difference` for R_{V_m} and $R_{I_{\text{ion}}}$, and `maxStateRelativeL2Difference` for the state maximum. A nonlinear iteration can stop only after `minNonlinearIterations`; convergence then requires `nonlinearTolerance`, `nonlinearVmTolerance` and `nonlinearIionTolerance`. The state criterion `nonlinearStatesTolerance` is included only when `nonlinearRequireStatesConvergence=true`.

6.2 Picard fixed-point method

The Picard branch treats the source as lagged during the PDE solve. At iteration k the states and ionic source are computed from $\mathbf{x}_k$, then the linear PDE

$$\mathbf{A} \mathbf{V}_* = \mathbf{r}(I_{\text{ion}}(\mathbf{x}_k)) \quad (35)$$

is solved with the Eigen sparse solver selected by `linearSolver`. The next candidate is relaxed,

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \omega(\mathbf{V}_* - \mathbf{x}_k), \quad \omega = \text{nonlinearRelaxation}. \quad (36)$$

After the update the solver re-evaluates states and I_{ion} at $\mathbf{x}_{k+1}$ and checks the residuals above. In this electro-activation solver, unlike a pure MMS Picard study, a failed Picard solve rolls back unless `nonlinearAcceptUnconverged=true`.

Advantages. Picard is simple, cheap per nonlinear iteration, and uses the same linear PDE solver as the classical implicit formulation. It is a useful baseline and is often adequate when Δt is small or the ionic source is weakly nonlinear over one time step.

Disadvantages. The source derivative with respect to V_m is ignored in the PDE matrix, so convergence can be slow near the action-potential upstroke or for large Δt . A small residual may require many iterations, and under-relaxation may be needed for robustness.

6.3 Diagonal I_{ion} linearisation

The `diagonalIion` branch is Picard plus a local finite-difference approximation of the source derivative. Define

$$S(V_m) = -I_{\text{ion}}(V_m, \mathbf{s}(V_m)) + I_{\text{ext}}/(\chi C_m).$$

The solver approximates, cell by cell,

$$S'_i(V_{m,k}) \approx -\frac{I_{\text{ion},i}(V_{m,k} + \varepsilon_d) - I_{\text{ion},i}(V_{m,k})}{\varepsilon_d}, \quad \varepsilon_d = \text{diagonalIionEpsilon}. \quad (37)$$

The linearised residual $\mathbf{A}\mathbf{x} - \theta S(\mathbf{x})$ gives the system

$$(\mathbf{A} - \theta \text{diag}(S'_k)) \mathbf{x}_{k+1} = \mathbf{r}(I_{\text{ion}}(\mathbf{x}_k)) - \theta \text{diag}(S'_k) \mathbf{x}_k. \quad (38)$$

The update can still be relaxed with `nonlinearRelaxation`, and the same convergence and rollback logic as Picard is used.

Advantages. It captures the dominant local stiffness of $I_{\text{ion}}(V_m)$ without building a full Jacobian. It often reduces the number of nonlinear iterations relative to Picard while preserving a sparse scalar PDE matrix.

Disadvantages. Only the cell-local diagonal sensitivity is included. Couplings through ODE states, high-order quadrature averaging and spatial reconstruction are not represented in the matrix. The finite-difference step `diagonalIionEpsilon` must be chosen above ODE/noise levels but small enough to be a useful derivative.

6.4 Jacobian-Free Newton–Krylov

JFNK applies Newton to (29):

$$J(\mathbf{x}_k)\boldsymbol{\delta}_k = -F(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \boldsymbol{\delta}_k,$$

but never assembles J . GMRES sees only matrix-vector products approximated by

$$J(\mathbf{x}_k) \mathbf{v} \approx \frac{F(\mathbf{x}_k + \varepsilon \mathbf{v}) - F(\mathbf{x}_k)}{\varepsilon}, \quad \varepsilon = \varepsilon_0 \frac{1 + \|\mathbf{x}_k\|_2}{\|\mathbf{v}\|_2}, \quad (39)$$

with $\varepsilon_0 = \text{jfnkEpsilon}$. Every evaluation of F re-runs the ODEs and recomputes I_{ion} from the same time-step-start state snapshot. This is expensive but gives a genuinely coupled nonlinear residual.

6.5 In-house restarted GMRES

`solveGMRES` implements GMRES(m) with modified Gram–Schmidt. It solves the Krylov least-squares problem for the Newton correction using at most `jfnkMaxKrylovIterations` basis vectors per restart and at most `jfnkMaxRestarts` restarts. The stopping test is controlled by `jfnkLinearTolerance`. The method is “matrix-free” here because the linear operator is the finite-difference shell in (39), not a stored sparse matrix.

Advantages. JFNK sees the full nonlinear dependence $V_m \mapsto \mathbf{s}(V_m) \mapsto I_{\text{ion}}(V_m, \mathbf{s})$ and usually has far better nonlinear convergence than Picard in stiff regimes. It also avoids explicitly forming a dense or complicated Jacobian.

Disadvantages. Each Krylov vector requires an additional residual evaluation, hence a new ODE integration at every cell or quadrature point. The method is sensitive to the finite-difference perturbation, ODE tolerances and line-search choices. In this Eigen implementation the JFNK inner GMRES has no preconditioner.

6.6 Armijo backtracking line search

The raw Newton update may increase the nonlinear residual when the linear model is poor. If `jfnkLineSearch=true`, the solver tries

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \boldsymbol{\delta}_k, \quad \alpha = \text{nonlinearRelaxation}, \alpha/2, \alpha/4, \dots$$

until the Armijo sufficient-decrease condition holds:

$$\|F(\mathbf{x}_k + \alpha \boldsymbol{\delta}_k)\|_2^2 \leq (1 - 2c\alpha) \|F(\mathbf{x}_k)\|_2^2, \quad c = \text{jfnkArmijoC}. \quad (40)$$

Backtracking stops after `jfnkLineSearchMaxIter` attempts or when the step length reaches `jfnkLineSearchAlphaMin`. This globalises Newton: large Newton steps are accepted when they reduce the residual, otherwise they are damped.

6.7 Initial guess, ODE clamp and rollback

The first guess for the time step is either $\mathbf{V}^n$ or the linear extrapolation

$$\mathbf{x}_0 = \mathbf{V}^n + \frac{\Delta t}{\Delta t_{\text{prev}}} (\mathbf{V}^n - \mathbf{V}^{n-1}), \quad (41)$$

selected by `jfnkInitGuessOrder`. It is clamped to `[jfnkInitGuessVmMin, jfnkInitGuessVmMax]`. The same bracket can also be applied to voltage values passed into the ODE solver through `jfnkClampODEInput`; this protects the ionic model during JFNK finite-difference probes.

If the nonlinear loop reaches its iteration budget without satisfying the convergence tests, `nonlinearAcceptUnconverged` decides whether the last iterate is committed. With the default `false`, the solver rolls back V_m , I_{ion} and all ionic states to the beginning of the time step. Rollback prevents a bad nonlinear solve from contaminating the remaining activation simulation.

7 Linear solver: Eigen back-end

The wrapper `solveSparseSystem` drives either of two Eigen sparse solvers on the explicit operator A (Picard / diagonal paths):

linearSolver key	Eigen type	Type
<code>SparseLU</code>	<code>Eigen::SparseLU</code>	Direct LU
<code>BiCGSTAB</code>	<code>Eigen::BiCGSTAB + IncompleteLUT</code>	Iterative

Both share the `tolerance` and `maxIterations` dictionary keys (the former is ignored by `SparseLU` since it is direct). The matrix-free JFNK branch never instantiates these wrappers; instead it calls the in-house GMRES (Section 5.2) on the FD Jacobian shell.

8 Ionic ODE integration

For each cell or cell QP, the TNNP ODE (3) is advanced by the OpenFOAM adaptive integrator selected via the `solver` key. Default RKF45 [6]. Keys read from `constant/timeIntegrationProperties`:

Key	Meaning	Default
<code>solver</code>	Concrete ODE integrator	RKF45
<code>absTol</code>	Absolute tolerance per state	10^{-9}
<code>relTol</code>	Relative tolerance per state	10^{-6}
<code>maxSteps</code>	Cap on sub-steps per outer call	10^4
<code>initialODEStep</code>	Initial sub-step (ms)	10^{-5}

`relTol` drives the noise floor of the matrix-free Jacobian: $\eta_F \sim \text{relTol}$. Tightening `relTol` buys a lower attainable outer tolerance at the cost of more RKF45 sub-steps per Newton iteration.

9 TNNP numerical protection

The master switch `TNNPNumericalProtection` (default `false`) gates a local guard invoked at every external entry into the embedded TNNP kernel (`calculateCurrent`, `solveODE.start`, `solveODE.end`, `derivatives`).

Key	Type	Default
<code>TNNPNumericalProtection</code>	Switch	<code>false</code>
<code>TNNPCLampGates</code>	Switch	<code>true</code>
<code>TNNPCLampVmForRates</code>	Switch	<code>true</code>
<code>TNNPLogNumericalProtection</code>	Switch	<code>true</code>
<code>TNNPConcentrationFloor</code>	scalar (mM)	10^{-12}
<code>TNNPVMinForRates</code>	scalar (mV)	-200
<code>TNNPVMaxForRates</code>	scalar (mV)	$+100$
<code>TNNPVoltageZeroEps</code>	scalar (mV)	10^{-8}
<code>TNNPMaxProtectionLogEntries</code>	label	10^6
<code>TNNPProtectionLog</code>	word	<code>TNNP_numericalProtection_summary.dat</code>

When protection is enabled, the guards applied in order are:

1. **Non-finite V** : `!std::isfinite(V) → reset to  $-86.2$  mV.`
2. **V clamp for rates** (if `TNNPCLampVmForRates`): clamp to `[TNNPVMinForRates, TNNPVMaxForRates]`.
3. **$V = 0$ singularity avoidance**: push V off zero by `TNNPVoltageZeroEps` (sign preserving) when $|V|$ falls below that threshold.

4. **Concentration floor** on K_i, Na_i, Ca_i, Ca_{SR} : non-finite or below-floor entries are reset to `TNNPConcentrationFloor`.
5. **Gate clamp** on $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ (if `TNNPCLampGates`): non-finite to zero, clamp to $[0, 1]$.

The embedded implementation stores a capped correction count and emits a compact summary file at exit; it does not depend on external ionic-model logging utilities.

10 Activation tracking and benchmarks

Activation time per cell. First-crossing of V_{thr} in a time step is recorded by linear interpolation:

$$w = \text{clip}\left(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n}, 0, 1\right), \quad t_{\text{act}} = t^n + w \Delta t, \quad (42)$$

with a per-cell flag cleared after the first crossing so subsequent re-crossings are ignored.

Early termination on P8. When `stopAfterPointActivation = true`, the cell nearest `stopActivationPoint` (P8 in the Niederer benchmark) is monitored and `effectiveEndTime` is set to $t_{\text{act}} + \text{stopDelayAfterActivation}$ once its $t_{\text{act}} > 0$.

Diagonal sampling. `sampleIDW` performs inverse-distance-weighted interpolation at `nDiagonalSamples` points between `diagonalStart` and `diagonalEnd`, with $k = \text{sampleNeighbours}$, $p = \text{samplePower}$. Because a diagonal sample averages its k nearest cells and unactivated cells contribute $t_{\text{act}} = 0$, a sample watched *live* climbs from below as the front fills its neighbourhood; the final value (all k neighbours activated) is the correct one.

Live activation output. `writeActivationSamples` is called once before the time loop (so `diagonalActivationTime` and `P8_activationTime.dat` exist from $t = 0$ with every point pending) and again whenever the activated-cell count increases, overwriting the files. The run can therefore be monitored as the front advances; the driver exposes the live file in each run directory by a symbolic link that is replaced by a real copy at the end.

Activation-failure detection. When `abortOnActivationFailure = true` the solver stops early (cleanly, exit 0) if the activation fails, avoiding a full run to `endTime`. Two criteria, using the running activated-cell count N_a and the time t_* of the last new activation:

- **no ignition:** $N_a = 0$ and $t > \text{activationIgnitionTimeout}$;
- **conduction block:** $N_a > 0$, the stop point is not yet reached, and $t - t_* > \text{activationStallTime}$.

The outcome is always recorded in `postProcessing/.../activationStatus.dat` as one of `OK`, `FAILED_NO_IGNITION`, `FAILED_CONDUCTION_BLOCK` or `INCOMPLETE` (with the final time and N_a). The driver reads this file and, on a failure, logs it to `activation_status_summary.dat` and continues with the next configuration rather than aborting the sweep.

11 Solver flow chart

The pseudo-code below follows the actual solver. The shared setup is common to all nonlinear methods; then one of the three method-specific blocks is executed according to `nonlinearMethod`.

Algorithm 1 Shared time-step setup and final commit/rollback.

```
1: Read dictionaries and assemble  $\mathbf{M}$ ,  $\mathbf{K}$ ,  $\mathbf{L} = \mathbf{K}/(\chi C_m)$ .
2: while  $t^n < T$  do
3:   Save snapshots:  $\mathbf{V}^n$ ,  $I_{\text{ion}}^n$ ,  $\mathbf{s}^n$ .
4:   Build  $\mathbf{A} = \mathbf{M}/\Delta t - \theta \mathbf{L}$  and  $\mathbf{B} = \mathbf{M}/\Delta t + (1 - \theta)\mathbf{L}$ .
5:   Build old source  $\mathbf{S}^n = -I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m)$ .
6:   if jfnkInitGuessOrder  $\geq 1$  and previous step exists then
7:      $\mathbf{x}_0 \leftarrow \mathbf{V}^n + (\Delta t/\Delta t_{\text{prev}})(\mathbf{V}^n - \mathbf{V}^{n-1})$ .
8:     Clamp  $\mathbf{x}_0$  to [jfnkInitGuessVmMin, jfnkInitGuessVmMax].
9:   else
10:     $\mathbf{x}_0 \leftarrow \mathbf{V}^n$ .
11:   end if
12:   Run Algorithm 2, 3 or 4.
13:   if not converged and nonlinearAcceptUnconverged is false then
14:     Roll back  $V_m, I_{\text{ion}}, \mathbf{s}$  to the saved  $t^n$  snapshots.
15:   else
16:     Commit the final nonlinear iterate.
17:   end if
18:   Update activation times and benchmark samples.
19: end while
```

Algorithm 2 Picard nonlinear solve.

Input: Initial candidate $\mathbf{x}_0$, state snapshot $\mathbf{s}^n$.

```
1: for  $k = 0, \dots, \text{nonlinearIterations} - 1$  do
2:   Store previous iterate  $\mathbf{x}_{k-1}$ ,  $I_{\text{ion},k-1}$ ,  $\mathbf{s}_{k-1}$  for residuals.
3:   EVALUATENONLINEARFIELDS( $\mathbf{x}_k$ ): reset states to  $\mathbf{s}^n$ , reconstruct  $V_m$  at Iion quadrature
   points if enabled, call ionicModel->solveODE, compute  $I_{\text{ion},k}$ .
4:   Build RHS  $\mathbf{r}(I_{\text{ion},k})$  using (30).
5:   Solve  $\mathbf{A}\mathbf{V}_* = \mathbf{r}(I_{\text{ion},k})$  with linearSolver.
6:   Relax:  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \text{nonlinearRelaxation}(\mathbf{V}_* - \mathbf{x}_k)$ .
7:   Re-evaluate ODEs and  $I_{\text{ion}}$  at  $\mathbf{x}_{k+1}$ .
8:   Compute  $R_{\text{cpl}}$ ,  $R_{V_m}$ ,  $R_{I_{\text{ion}}}$  and optional  $R_{\mathbf{s}}$ .
9:   if all requested criteria pass and  $k + 1 \geq \text{minNonlinearIterations}$  then
10:    Mark converged and exit loop.
11:   end if
12: end for
```

Algorithm 3 diagonalIion nonlinear solve.

Input: Initial candidate $\mathbf{x}_0$, state snapshot $\mathbf{s}^n$.

```
1: for  $k = 0, \dots, \text{nonlinearIterations} - 1$  do
2:   Evaluate states and  $I_{\text{ion},k}$  at  $\mathbf{x}_k$  exactly as in Picard.
3:   Approximate the local derivative  $S'_k \approx -[I_{\text{ion}}(\mathbf{x}_k + \varepsilon_d) - I_{\text{ion}}(\mathbf{x}_k)]/\varepsilon_d$ , with  $\varepsilon_d =$ 
   diagonalIionEpsilon.
4:   Assemble  $\mathbf{A}_k = \mathbf{A} - \theta \text{diag}(S'_k)$ .
5:   Build corrected RHS  $\mathbf{r}_k = \mathbf{r}(I_{\text{ion},k}) - \theta \text{diag}(S'_k)\mathbf{x}_k$ .
6:   Solve  $\mathbf{A}_k\mathbf{V}_* = \mathbf{r}_k$  with linearSolver.
7:   Relax, re-evaluate ODEs/ $I_{\text{ion}}$ , compute residuals and test convergence as in Picard.
8: end for
```

Algorithm 4 Jacobian-Free Newton–Krylov solve.

Input: Initial candidate $\mathbf{x}_0$, state snapshot $\mathbf{s}^n$.

```
1: for  $k = 0, \dots, \text{nonlinearIterations} - 1$  do
2:   Evaluate  $\mathbf{F}_k = \mathbf{F}(\mathbf{x}_k)$ ; this resets states to  $\mathbf{s}^n$ , solves ODEs and recomputes  $I_{\text{ion}}$ .
3:   Compute residuals. If all requested criteria pass, mark converged and exit.
4:   Define matrix-free product  $\mathcal{J}_k \mathbf{v} = [\mathbf{F}(\mathbf{x}_k + \varepsilon \mathbf{v}) - \mathbf{F}_k]/\varepsilon$ , with  $\varepsilon$  from jfnkEpsilon.
5:   Solve  $\mathcal{J}_k \boldsymbol{\delta}_k = -\mathbf{F}_k$  by restarted GMRES using jfnkMaxKrylovIterations, jfnkMaxRestarts and
   jfnkLinearTolerance.
6:   if jfnkLineSearch then
7:     Start  $\alpha = \text{nonlinearRelaxation}$  and backtrack until Armijo (40) succeeds, the trial budget
     is exhausted, or jfnkLineSearchAlphaMin is reached.
8:   else
9:      $\alpha \leftarrow \text{nonlinearRelaxation}$ .
10:  end if
11:   $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha \boldsymbol{\delta}_k$ .
12:  Re-evaluate  $\mathbf{F}(\mathbf{x}_{k+1})$ , update fields, compute residuals and test convergence.
13: end for
```

Legend. `Blue` = Python constant set in `run_convergence.py`; `green` = OpenFOAM dictionary key read by the solver. Constants and keys are mapped one-to-one in section 12.

12 Manufactured driver: `run_convergence.py`

The convergence script `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials.electro/highOrderManufacturedFDAImp` generates an MMS case whose numerical path is intended to match this solver. The tables below describe the constants exposed there, the OpenFOAM dictionary entries they generate, and which part of the discrete equations they affect. The same dictionary keys are read by `highOrderElectroActivationFoamImplicitJfNK`; only problem-specific items such as exact fields versus TNNP/stimulus differ.

12.1 Geometry and parameter sweep

Python constant	OpenFOAM key	Affected equation / mechanism
DIMENSIONS	—	Selects 2D or 3D manufactured domain and the corresponding LRE stencils in <code>P_LEVELS_BY_DIM</code> .
N_VALUES	<code>blockMeshDict</code> cell counts	Spatial refinement. This changes FV/LRE discretisation error in the PDE, Iion quadrature and state reconstruction.
DT_VALUES	<code>controlDict.deltaT</code>	Outer PDE time step. Also sets the integration interval for every local ODE solve.
END_TIME	<code>controlDict.endTime</code>	Final time of the transient MMS test.
IMPLICIT_SCHEMES	<code>spatialIntegrationProperties</code> <code>implicitScheme3</code>	Choices of schemes: <code>backwardEuler</code> ($\theta = 1$) or <code>crankNicolson</code> ($\theta = 1/2$).
MASS_MATRIX_VALUES	<code>spatialIntegrationProperties</code> <code>massMatrix</code>	Choices of mass matrix: <code>trapezoidal</code> or LRE-consistent <code>M</code> in $\mathbf{A} = \mathbf{M}/\Delta t - \boldsymbol{\theta} \mathbf{L}$.

Python constant	OpenFOAM key	Affected equation / mechanism
ACTIVE_HO_PAIRS_Vm_States	useHighOrder_Vm useHighOrder_Iion LRECoeffs_*	/ Tuple (Vm-level, Iion/states-level). Vm level affects diffusion and consistent mass; Iion/states level affects where ODEs are solved and where the nonlinear source is integrated.
P_LEVELS_BY_DIM	highOrderCoeffs	Maps p1,p2,p3 to LRE polynomial order N, stencil target Nn, radial weight exponent k, and maxStencilSize.

12.2 Nonlinear method and convergence controls

Python constant	OpenFOAM key	Affected equation / mechanism
NONLINEAR_METHOD	nonlinearMethod	Selects the nonlinear strategy for the PDE/source coupling: Picard, JFNK, or diagonalIion.
IMPLICIT_NONLINEAR_ITERATIONS	nonlinearIterations	Maximum number of nonlinear corrections per PDE time step.
IMPLICIT_MIN_NONLINEAR_ITERATIONS	minNonlinearIterations	Minimum number of corrections before a convergence test may stop the loop.
NONLINEAR_TOLERANCE	nonlinearTolerance	Coupled PDE residual tolerance R_{cpl} . Used directly by JFNK and diagonalIion; in the electro solver it is also reported for Picard.
NONLINEAR_VM_TOLERANCE	nonlinearVmTolerance	Relative correction of the PDE unknown V_m .
NONLINEAR_STATES_TOLERANCE	nonlinearStatesTolerance	Relative correction of all ODE states. In the MMS solver these are u_1, u_2, u_3 ; in this electro solver these are the TNNP state variables.
NONLINEAR_REQUIRE_STATES_CONVERGENCE	nonlinearRequireStatesConverged	If true, the state residual is part of the stopping criterion.
NONLINEAR_ACCEPT_UNCONVERGED	nonlinearAcceptUnconverged	If false, a failed electro nonlinear solve rolls back V_m, I_{ion} and states to t^n .
NONLINEAR_IION_TOLERANCE	nonlinearIionTolerance	Relative correction of the nonlinear source I_{ion} .
NONLINEAR_RELAXATION	nonlinearRelaxation	Picard/diagonal mixing factor or initial Newton step length for JFNK/Armijo.

12.3 JFNK and diagonal-Iion controls

Python constant	OpenFOAM key	Affected equation / mechanism
JFNK_MAX_KRYLOV_ITERATIONS	jfnkMaxKrylovIterations	Restart length m of GMRES(m), controlling memory and Krylov approximation quality.
JFNK_MAX_RESTARTS	jfnkMaxRestarts	Number of times the GMRES basis may be discarded and rebuilt.
JFNK_LINEAR_TOLERANCE	jfnkLinearTolerance	Relative tolerance of the Newton correction solve $J\delta = -F$.
JFNK_EPSILON	jfnkEpsilon	Base perturbation for matrix-free Jacobian-vector products. It should be above the ODE/noise floor but small enough to approximate a derivative.
JFNK_INIT_GUESS_ORDER	jfnkInitGuessOrder	Chooses lagged or linearly extrapolated initial V_m for the nonlinear solve.
JFNK_INIT_GUESS_VM_MIN/MAX	jfnkInitGuessVmMin/Max	Clamp interval for extrapolated guess and, if enabled, ODE inputs.
JFNK_CLAMP_ODE_INPUT	jfnkClampODEInput	Protects the ODE solve from unphysical JFNK finite-difference probes.
JFNK_LINE_SEARCH	jfnkLineSearch	Turns Armijo backtracking on/off.
JFNK_LINE_SEARCH_MAX_ITER	jfnkLineSearchMaxIter	Maximum number of halvings of the Newton step.
JFNK_LINE_SEARCH_ALPHA_MIN	jfnkLineSearchAlphaMin	Floor for the Armijo step length.
JFNK_ORMIJO_C	jfnkArmijoC	Sufficient-decrease parameter in (40).
DIAGONAL_IION_EPSILON	diagonalIionEpsilon	Finite-difference step for the diagonal source derivative in <code>diagonalIion</code> .

12.4 Linear PDE solver

Python constant	OpenFOAM key	Affected equation / mechanism
IMPLICIT_LINEAR_SOLVER	linearSolver	Eigen sparse solver used by Picard and <code>diagonalIion</code> for $AV = r$. JFNK instead uses GMRES on the Jacobian-free shell.
IMPLICIT_TOLERANCE	tolerance	BiCGSTAB tolerance for the linear PDE solve.
IMPLICIT_MAX_ITERATIONS	maxIterations	BiCGSTAB iteration cap.

12.5 State ODE integrator

Python constant	OpenFOAM key	Affected equation / mechanism
STATE_ODE_SOLVER	<code>stateODESolver</code> <code>timeIntegrationProperties.solve</code>	/ Local ODE integrator for MMS states or TNNP states.
STATE_ODE_ABS_TOL	<code>stateODEAbsTol</code> <code>timeIntegrationProperties.absTol</code>	/ Absolute tolerance of the ODE solve.
STATE_ODE_REL_TOL	<code>stateODERelTol</code> <code>timeIntegrationProperties.relTol</code>	/ Relative ODE tolerance. This controls the noise floor seen by JFNK finite differences.
STATE_ODE_INITIAL_STEP	<code>stateODEInitialStep</code> <code>initialODEStep</code>	/ First RKF45 sub-step attempted inside one PDE time step.
STATE_ODE_MAX_STEPS	<code>stateODEMaxSteps</code> / <code>maxSteps</code>	Maximum ODE substeps per local solve.

12.6 LRE high-order parameters per p-level

`P_LEVELS_BY_DIM[dim][p]` gives, for each (dim, p): N (polynomial order), N_n (stencil neighbours), k (search radius), `maxStencilSize` (cap). The script writes these into a nested `highOrderCoeffs{LRECoeffs.Vm{...}; LRECoeffs.Iion{...}}` block whenever HO is enabled for either V_m or I_{ion} .

12.7 Diagnostics and run-control

Python constant	OpenFOAM key	Affected mechanism
<code>WRITE_NONLINEAR_RESIDUALS</code>	<code>writeNonlinearResiduals</code>	Residual log on/off.
<code>KEEP_LOGS_ON_SUCCESS</code>	—	Python-only switch: keep solver/blockMesh logs after successful runs. Failed-run logs are always kept.

For the electro-activation tutorial the analogous driver also exposes activation-threshold, P8 stopping and diagonal-sampling parameters. They do not affect the PDE/ODE solve itself; they affect only stopping and post-processing of activation times.

13 Results

This section collects the current convergence results for the 2D and 3D Niederer-style benchmarks, run with the three nonlinear strategies exposed by the solver (`Picard`, `diagonalIion`, `JFNK`). All runs use the Crank–Nicolson time scheme ($\theta = 1/2$), the consistent high-order mass matrix when LRE is enabled, the RKF45 state ODE integrator, and a single requested $\Delta t_{\text{req}} = 10^{-4}$ s. Six LRE pairs (V_m, I_{ion}) are sampled: `N0/N0`, `N0/p1`, `N0/p3`, `p3/N0`, `p3/p1`, `p3/p3`. Each table lists, for the metric in its caption, the value of every available run; NC marks combinations that have not been run yet. The 2D results live under `highOrderNiedererEtAl2012ImplicitJfNK[_diagonalIion|_Picard]/results/example0/2D/`; the 3D results under the same trees but with the `3D/` suffix. Wall-clock time is the elapsed time reported by `/usr/bin/time -v`, rounded to integer minutes. Peak RSS is the maximum resident set size of the solver process, expressed in MiB (`[kB/1024]`).

13.1 2D Niederer-style benchmark

The 2D case follows the 20×8 mm planar reduction used by the explicit and non-JfNK implicit reports (single cell in z , planar fibres), exercised at mesh sizes $\Delta x \in \{0.5, 0.25, 0.125, 0.1\}$ mm; only the `Picard` sweep currently includes the finest mesh.

Table 7: 2D P8 activation time [ms] at $\Delta t = 10^{-4}$ s. Columns are mesh sizes Δx in mm.

Vm	Iion	0.5	0.25	0.125	0.1
<i>Picard</i>					
NO	NO	153.32	62.99	45.03	42.76
NO	p1	91.05	54.86	43.21	41.84
NO	p3	32.46	39.14	39.83	NC
p3	NO	180.55	68.84	48.83	NC
p3	p1	68.92	58.81	46.33	NC
p3	p3	33.09	40.22	40.84	NC
<i>diagonalIion</i>					
NO	NO	153.16	62.86	44.92	NC
NO	p1	90.92	54.74	43.08	NC
NO	p3	32.37	38.97	39.66	NC
p3	NO	180.42	68.71	48.67	NC
p3	p1	68.83	58.70	46.20	NC
p3	p3	33.00	40.03	40.65	NC
<i>JFNK</i>					
NO	NO	153.13	62.85	44.87	NC
NO	p1	90.92	54.72	NC	NC
NO	p3	32.36	38.92	39.59	NC
p3	NO	180.38	68.69	NC	NC
p3	p1	68.83	NC	NC	NC
p3	p3	33.00	NC	NC	NC

Table 8: 2D solver wall-clock time [min] at $\Delta t = 10^{-4}$ s (rounded to integer minutes).

Vm	Iion	0.5	0.25	0.125	0.1
<i>Picard</i>					
NO	NO	15	31	83	125
NO	p1	20	51	141	197
NO	p3	23	102	402	NC
p3	NO	18	40	112	NC
p3	p1	20	60	176	NC
p3	p3	22	105	419	NC
<i>diagonalIion</i>					
NO	NO	18	31	81	NC
NO	p1	22	52	147	NC
NO	p3	29	131	533	NC
p3	NO	21	38	107	NC
p3	p1	23	59	164	NC
p3	p3	28	133	549	NC
<i>JFNK</i>					
NO	NO	59	155	544	NC
NO	p1	82	263	NC	NC
NO	p3	124	647	3407	NC
p3	NO	67	169	NC	NC
p3	p1	87	NC	NC	NC
p3	p3	131	NC	NC	NC

Table 9: 2D peak resident set size [MiB] at $\Delta t = 10^{-4}$ s.

Vm	Iion	0.5	0.25	0.125	0.1
<i>Picard</i>					

Vm	lion	0.5	0.25	0.125	0.1
NO	NO	148	152	174	194
NO	p1	149	157	191	219
NO	p3	155	177	288	NC
p3	NO	155	191	346	NC
p3	p1	156	194	358	NC
p3	p3	161	218	457	NC
<i>diagonalIion</i>					
NO	NO	148	153	176	NC
NO	p1	149	158	194	NC
NO	p3	155	181	296	NC
p3	NO	155	191	347	NC
p3	p1	156	195	365	NC
p3	p3	161	218	457	NC
<i>JFNK</i>					
NO	NO	148	153	173	NC
NO	p1	149	158	NC	NC
NO	p3	155	181	299	NC
p3	NO	152	187	NC	NC
p3	p1	153	NC	NC	NC
p3	p3	160	NC	NC	NC

Discussion of the 2D results. At fixed $(\Delta x, \Delta t, \text{LRE pair})$ the three nonlinear strategies give activation times that agree to within $\sim 0.1\text{--}0.2$ ms; e.g. at $\Delta x = 0.125$ mm with (NO, NO) the values are 45.03/44.92/44.87 ms for Picard, diagonalIion, JFNK respectively. The 2D nonlinear-coupling choice is therefore essentially *free* from the accuracy standpoint here — the relevant errors are spatial and temporal, not nonlinear-iteration errors. The three methods differ sharply in cost: JFNK is $\sim 3\text{--}6\times$ more expensive than Picard for the same case at $\Delta x = 0.5$ mm and grows faster than linearly with mesh size, because every Krylov matrix–vector product performs a full ODE re-evaluation ($\Delta x = 0.125$, (NO, p3): 402 min for Picard, 533 min for diagonalIion, 3407 min — ~ 2.4 days — for JFNK). Peak RSS is dominated by the LRE stencil and is essentially method-independent; the JFNK column is slightly lower for some entries because the matrix-free shell avoids the diagonal correction and the explicit BiCGSTAB workspace.

13.2 3D Niederer-style benchmark

The 3D sweep follows the published Niederer slab geometry ($20 \times 3 \times 7$ mm, $40 \times 6 \times 14$ cells at $\Delta x = 0.5$ mm). At the time of writing only the diagonalIion and Picard sweeps include 3D runs, all at $\Delta x = 0.5$ mm. The JFNK 3D sweep has not yet been run.

Table 10: 3D P8 activation time [ms] at $\Delta x = 0.5$ mm, $\Delta t = 10^{-4}$ s. Columns are the available nonlinear methods.

Vm	lion	Picard	diagonalIion
NO	NO	136.67	136.61
NO	p1	33.50	33.39
NO	p3	30.59	30.51
p3	NO	149.13	149.06
p3	p1	32.44	32.36
p3	p3	29.63	NC

Table 11: 3D solver wall-clock time [min] at $\Delta x = 0.5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	Picard	diagonalIion
NO	NO	80	82
NO	p1	168	229
NO	p3	1489	2159
p3	NO	163	143
p3	p1	167	269
p3	p3	1487	NC

Table 12: 3D peak resident set size [MiB] at $\Delta x = 0.5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	Picard	diagonalIion
NO	NO	156	157
NO	p1	211	211
NO	p3	787	849
p3	NO	1123	1121
p3	p1	1164	1162
p3	p3	1565	NC

Discussion of the 3D results. On the coarse 3D slab ($\Delta x = 0.5$ mm), activating LRE on the ionic source alone changes the activation time from ≈ 136.7 ms (NO/NO) to ≈ 33.5 ms (NO/p1) and ≈ 30.6 ms (NO/p3) — the same qualitative high-order-on-source effect already seen in the 2D sweep and in the explicit/implicit Niederer reports. The fully high-order (p3/p3) reaches 29.63 ms with Picard (the diagonalIion counterpart has not yet been completed). The two nonlinear methods again agree to better than 0.1 ms for every entry where both are available; Picard is generally faster, with the most striking gap on the heaviest case (NO/p3: 1489 min for Picard vs. 2159 min for diagonalIion). Peak RSS scales with the LRE stencil size and is essentially method-independent. The dataset is too thin in Δx to draw quantitative conclusions about mesh convergence in 3D; the immediate gap to fill is the missing p3/p3 diagonalIion entry and a JFNK 3D sweep at the same mesh.

A Dimension table

Field	OpenFOAM dimensionSet	SI
Vm	[1, 2, -3, 0, 0, -1, 0]	volt
Iion	dimVoltage/dimTime	V s^{-1}
externalStimulusCurrent	dimCurrent/dimVolume	A m^{-3}
conductivity	dimCurrent^2 dimTime^3 / (dimMass dimVolume)	S m^{-1}
chi	dimless/dimLength	m^{-1}
Cm	dimCurrent dimTime / (dimVoltage dimArea)	F m^{-2}

B File map

Path (relative to cardiacFoam-pc/)	Role
<code>applications/solvers/.../JfNK.C</code>	Main translation unit, including the embedded TNNP CellML kernel, state update, current evaluation, and local ODE integration.
<code>applications/solvers/.../createFields.H</code>	Field initialisation and dictionary reading.
<code>tutorials_electro/.../run_convergence.py</code>	Driver script.
<code>constant/cardiacProperties</code>	χ , C_m , $\mathbf{D}$, ionic model.
<code>constant/timeIntegrationProperties</code>	ODE solver, tolerances, sub-step cap.
<code>constant/spatialIntegrationProperties</code>	Time scheme, HO flags, JFNK / linear / line-search options.
<code>constant/stimulusProtocol</code>	Stimulus regions and timing.

C Differences vs. the PETSc sibling

Algorithmically the two solvers are identical. The only differences are in the linear-algebra back-end and a few user-visible knobs:

Aspect	This solver (Eigen)	PETSc sibling
Sparse matrix container	<code>Eigen::SparseMatrix</code>	PETSc Mat
Sparse direct solver	<code>Eigen::SparseLU</code>	KSPPREONLY+PCLU
Iterative solver (Picard branch)	<code>Eigen::BiCGSTAB+ILUT</code>	KSP+PC (full PETSc catalogue)
JFNK inner Krylov	in-house GMRES	KSPGMRES on MatShell
Preconditioner inside JFNK	none (Krylov-only)	any PETSc PC
Build-time dependency	header-only Eigen3	PETSc + MPI
Parallel run support	serial only	MPI-parallel out of the box

All convergence-related parameters (`nonlinearTolerance`, `jfnkEpsilon`, `jfnkLineSearch*`, `jfnkInitGuess*`, `jfnkClampODEInput`, `nonlinearAcceptUnconverged`, `ODE_*`) have the same names and meanings in both solvers, so the same `run_convergence.py` configuration produces an algorithmically equivalent run with either back-end (modulo the performance and parallel-scaling differences).

References

- [1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, AJP – Heart and Circulatory Physiology **286** (2004) H1573–H1589.
- [2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, Prog. Biophys. Mol. Biol. **104** (2012) 22–48.
- [3] D. A. Knoll, D. E. Keyes, *Jacobian-Free Newton-Krylov methods: a survey of approaches and applications*, J. Comp. Phys. **193** (2004) 357–397.
- [4] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006.
- [5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, 2003.
- [6] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.